

Comprehensive Problem Set

Lectures 1–3: the compiler pipeline, tokens and regular expressions, lexer generation

Compiler Construction Module

Quizzes, understanding questions, and worked problems, with solutions

How to use this set

Three tiers, in increasing depth.

- **Part A: Quizzes.** Eighteen multiple-choice questions, six per lecture. Two minutes each. They test whether you have the facts straight.
- **Part B: Understanding questions.** Twelve short-answer questions. Five minutes each. They test whether you can say *why*, not just *what*. A good answer is three to six sentences.
- **Part C: Problems.** Ten exam-style problems with several parts each. Fifteen to twenty-five minutes each. They test whether you can *do* it: tokenise by hand, write the regex, run the constructions, trace the algorithm, write the proof.

Solutions for all three parts start on page 8. Do not look until you have committed to an answer; the point of a wrong answer is finding out *why* it was wrong.

Part A: Quizzes

Quiz: Lecture 1: the compiler and computational theory

- A1.** Which phase turns the characters `sum = sum + 1` into `ID ASSIGN ID PLUS INT`?
(a) parser (b) lexer (c) semantic analysis (d) code generator
- A2.** “Every variable is declared before it is used” is checked by:
(a) the lexer (b) the parser, through the grammar (c) semantic analysis, with a symbol table (d) the linker
- A3.** The parser recognises languages of which Chomsky type?
(a) Type 3 (b) Type 2 (c) Type 1 (d) Type 0
- A4.** “Does this loop terminate on every input?” is:
(a) decidable in linear time (b) decidable but PSPACE-complete

- (c) undecidable (d) a regular-language question
- A5.** The main reason the -O2 loop body has fewer memory instructions than -O0 is:
 (a) fewer iterations (b) variables kept in registers (c) the array is cached (d) shorter encodings
- A6.** A *sound* optimiser is one that:
 (a) always finds the fastest program (b) never changes the program's meaning (c) never misses an optimisation (d) is allowed to be wrong on rare inputs

Quiz: Lecture 2: tokens and regular expressions

- A7.** In most languages, whitespace:
 (a) is a punctuation token (b) separates tokens and is then discarded (c) is passed to the parser (d) is a lexical error
- A8.** A spec has a signed integer rule `INT = "-"?[0-9]+` and an operator rule `"-"`. The input `x-1` lexes as:
 (a) `ID INT(-1)` (b) `ID MINUS INT(1)` (c) it depends on rule order (d) a lexical error
- A9.** Rules `"<"`, `"<="`, `"<="`, `"<="` exist. On `a<=b`, the token after `ID(a)` is:
 (a) `<` (b) `<=` (c) `<=` (d) `<=`
- A10.** The regular expression `[0-9]*` matches the empty string.
 (a) true (b) false
- A11.** Which of these is *not* a regular language?
 (a) identifiers (b) comments that nest to any depth (c) hexadecimal literals (d) string literals without escapes
- A12.** A hand-written lexer calls `read()` and only then discovers that the character does not belong to the current token. The standard fix is:
 (a) a bigger buffer (b) `peek()` before `read()`, or unread the character (c) rewrite it as a regex (d) recursion

Quiz: Lecture 3: automating lexical analysis

- A13.** In Thompson's construction, which rule adds *no* new states?
 (a) symbol (b) union (c) star (d) concatenation
- A14.** Subset construction on an n -state NFA yields a DFA with at most:
 (a) n states (b) n^2 states (c) 2^n states (d) $n!$ states
- A15.** Two DFA states are *distinguishable* when:
 (a) they have different names (b) some string is accepted from one but not the other (c) one is reachable and the other is not (d) they have different out-degree
- A16.** In the scan-and-backtrack lexer, the stack is cleared:
 (a) at the start of every token (b) whenever the DFA leaves an accepting state (c) whenever the DFA gets stuck (d) never; it grows for the whole file
- A17.** Rule order in the specification decides the token when:
 (a) always (b) never; longest match decides everything (c) two

- rules match the same longest prefix (d) the input is a keyword
- A18.** In Reps' fix, `hopeless[q][i] = true` means:
- (a) state q is not accepting
 - (b) position i is past the end of input
 - (c) reading ahead from state q at position i reaches no accepting state before getting stuck
 - (d) the token at position i has already been returned

Part B: Understanding questions

Understanding: Answer each in three to six sentences

- B1.** Why is the parser not run directly on the character stream? Give two concrete benefits of having a separate lexer.
- B2.** Why do lexers treat `-3` as two tokens (`MINUS`, `INT(3)`) rather than one signed literal? What goes wrong if the lexer produces signed literals?
- B3.** Keyword rules must be listed *before* the identifier rule in a spec, yet the order of the keyword rules *among themselves* does not matter. Explain both facts with the same principle.
- B4.** Explain why a lexer needs at least one character of lookahead, with an example where the lexer cannot decide the current token without seeing the next character.
- B5.** The NFA-to-DFA conversion can be exponential. Why is that acceptable for a lexer generator when it would be unacceptable in a regex library that matches user-supplied patterns?
- B6.** What is the error state of a DFA, why do we make δ total by adding it, and what role does “arriving in the error state” play in the lexer’s driver loop?
- B7.** “Read until the DFA gets stuck, then return what you have read” is not a correct tokeniser. Give a specification and an input on which it returns the wrong token, and say what the correct algorithm does differently.
- B8.** Balanced parentheses cannot be described by a regular expression. Explain why in terms of what a finite automaton can remember, and say which compiler phase handles them and with what kind of machine.
- B9.** Distinguish a token’s *text* (lexeme) from its *value* (attribute), with two examples where they differ. Why does the parser want the value rather than the text?
- B10.** The lexer decides that `if` is a keyword; the parser does not. Why is this the right division of labour, and what would the parser have to do if the lexer returned `ID("if")`?
- B11.** What does it mean for an optimiser’s analysis to be *sound* and *conservative*? Give one question the optimiser would like to answer exactly, why it cannot, and what it answers instead.

B12. Give a situation in which a lexer needs *states* (start conditions such as JFlex's <COMMENT>) rather than a single list of rules, and explain what a state changes about how rules are applied.

Part C: Problems

Problem: C1: tokenise by hand, with positions

Specification, in this order: "if"→IF, "else"→ELSE, ">="→GE, "="→ASSIGN, "-"→MINUS, "("→LP, ")"→RP, ";"→SEMI, [0-9]+→INT (value: the number), [a-z][a-z0-9]*→ID (value: the text), [\t]+→skip.

Input line 1: if (n>=10) x = x-1;

1. List every token in order with its class, its value, and its (line, column) start position, 1-based.
2. Two places in this line require the lexer to look at a character beyond the token it eventually returns. Identify both and say which character is looked at.
3. Suppose the ID rule were listed *before* the IF rule. Which tokens change?
4. Suppose the input had been x -= 1; (no spaces around the sign, a space after =). What tokens result, and would the parser accept it as an assignment of -1? Explain.

Problem: C2: write the regular expressions

Use JFlex syntax with `digit = [0-9]`.

1. Write `float` for floating-point literals in these forms: 3.14, 3., .5, 2e10, 6.02e+23, 1.5E-3. The forms 3, e5, ., and 1.e must *not* be single float tokens.
2. Write `string` for double-quoted string literals that may contain escape sequences `\n`, `\t`, `\"`, `\\`, and may not span lines.
3. Write `linecomment` for `//` comments.
4. Write `ident` for identifiers that may contain underscores and digits but may not start with a digit, and may not consist *only* of underscores.
5. For each of (a) and (b), give one string that the lexer will *split into two tokens* rather than reject, and say what the two tokens are.

Problem: C3: hand-written lexer bugs

A student writes this number lexer (Java-like; `input.read()` returns the next character and advances, or `-1` at end of input):

```
int c = input.read();
StringBuilder sb = new StringBuilder();
while (c >= '0' && c <= '9') {
    sb.append((char) c);
}
```

```

    c = input.read();
}
return new Token(INT, Integer.parseInt(sb.toString()));

```

1. Trace it on the input 12+3 and list the tokens the whole lexer produces (assume the operator and number lexers are called in turn). What is lost?
2. Name the bug in one phrase and fix the code using a one-character `peek()` that does not advance.
3. The same student lexes `=` and `==` with: *read a character; if it is =, return ASSIGN*. What does `a==b` become? Fix it, and state the general principle you applied.
4. Explain why an unterminated string literal made the student's string lexer loop forever, and what the loop must check.

Problem: C4: a broken specification

A JFlex spec lists, in this order:

```

{ident}      { return ID; }          // ident = [a-zA-Z_][a-zA-Z0
           -9_]*
"if"         { return IF; }
{intlitt}    { return INT; }         // intlitt = [0-9]+
"."          { return DOT; }
{ws}         { /* skip */ }

```

1. Give the token stream for the input `if iffy 007 3.5`, with values.
2. Three things are wrong or surprising. Identify them and fix the spec (you may add rules and reorder).
3. After your fix, give the token stream for the same input.
4. Add a rule for the keyword `else` and explain where it must go.

Problem: C5: the full pipeline on a small regular expression

Take $R = a(b \mid c)^*$ over $\Sigma = \{a, b, c\}$.

1. Apply Thompson's construction. List the rules in order with the state count each adds, draw the NFA, and give the total.
2. Run subset construction. Give every reachable DFA state as a set of NFA states, mark the accepting ones, and give the transition table (include the error state).
3. Minimise the DFA with the table-filling algorithm. Show the marks and the final machine.
4. Now take $L_2 =$ strings over $\{a, b\}$ whose second-to-last symbol is a . Give a 3-state NFA, run subset construction, and show that the resulting DFA has 4 states. Then prove that *no* DFA for L_2 has fewer than 4 states.

Problem: C6: minimisation with distractors

A DFA over $\{0, 1\}$ has start state A , accepting states $\{C, C'\}$, and transitions

	0	1
A	B	A'
A'	B	A
B	B	C
C	B	A'
C'	B	A
U	U	C'

1. Which states are unreachable from A ? Remove them.
2. Run the table-filling algorithm on the remaining states. Show every pair, the rule that marks it, and a witness string for each marked pair.
3. Give the minimal DFA and describe its language in words.

Problem: C7: the combined lexer DFA

Specification, in this order: $\text{"in"} \rightarrow \text{IN}$, $\text{"int"} \rightarrow \text{INTKW}$, $[\text{a-z}]^+ \rightarrow \text{ID}$, $[\text{0-9}]^+ \rightarrow \text{NUM}$, $\text{"<"} \rightarrow \text{SHL}$, $\text{"<="} \rightarrow \text{LE}$, $\text{"<"} \rightarrow \text{LT}$, $\text{"="} \rightarrow \text{ASSIGN}$.

1. Draw the (already minimised) combined DFA, with each accepting state labelled by *all* the rules that accept there and the one that wins.
2. For each input, give the token stream by tracing the DFA with longest match and priority: `in`, `int`, `integer`, `i`, `in7`, `<=`, `<=<`, `<<`.
3. The order of `IN` and `INTKW` in the spec is irrelevant. The order of `SHL`, `LE` and `LT` is also irrelevant. Explain both, and give a pair of rules whose relative order *would* matter.

Problem: C8: scan-and-backtrack and the quadratic trap

Rules: $R_1 = \text{ab}$, $R_2 = (\text{ab})^* \text{c}$. Input: `ababab` ($n = 6$ characters, no `c`).

1. Run the scan-and-backtrack algorithm from position 1. Show the stack after every step, where Phase 1 stops, how far Phase 2 backs up, and the returned token.
2. How many characters does the lexer *read* (Phase 1 iterations) to produce the first token? The second? The third? Total for the file?
3. Generalise to input $(\text{ab})^k$, $n = 2k$: give the total number of characters read as a function of k and its order in n .
4. Now add Reps' `hopeless` table. Which (q, i) pairs are marked while producing the first token? Trace the second token and count its reads. Give the total reads for $(\text{ab})^k$ with the fix.

Problem: C9: what the optimiser may and may not do

For each proposed rewrite, say whether it is *sound* (preserves the meaning of every program) for C-like integer code, and if not, give a program on which it changes the result.

1. $\text{x} + 0 \rightarrow \text{x}$

2. $x * 0 \rightarrow 0$, where x may be any expression
 3. $x / 2 \rightarrow x \gg 1$ for a signed `int x`
 4. $f() + f() \rightarrow 2 * f()$
 5. Deleting the body of `if (0) { ... }`
 6. Reordering the two statements `*p = 1; *q = 2;`
 7. Replacing `while (n > 0) { s += n; n--; }` by `s += n*(n+1)/2; n = 0;` when n is known to be non-negative at loop entry
- Then answer: which of the unsound rewrites would become sound if the optimiser could prove one extra fact, and what fact?

Problem: C10: decidability, and what the compiler does instead

Define `RETURNS-ZERO` = $\{\langle f \rangle : f \text{ is a function that returns 0 on at least one input}\}$.

1. Prove that `RETURNS-ZERO` is undecidable by reduction from $\text{HALT}_{TM} = \{\langle M, w \rangle : M \text{ halts on } w\}$. Give the construction, prove both directions of the “iff,” and derive the contradiction.
2. A compiler wants to replace every call `f(x)` by the constant 0 when `f` always returns 0. Is “`f` always returns 0” decidable? Reduce or decide.
3. Describe what constant propagation actually does with a function like

```
int f(int x) { if (x > 5) return 0; return g(x) - g(x); }
```

and why it may fail to conclude that `f` returns 0 even though, for total side-effect-free `g`, it does.

4. Place the following tasks in the Chomsky hierarchy (Type 3, 2, 1 or 0), each with one sentence of justification: scanning identifiers; matching brackets; checking that a function is called with the right number of arguments; deciding whether a variable is ever read after the last write to it; recognising string literals with escapes; rejecting nested block comments deeper than 3.

Solutions

Part A: quiz answers

Solution: Lecture 1

A1 (b). The lexer produces tokens; the parser consumes them.

A2 (c). Declaration before use relates two occurrences of the same name at unbounded distance; a context-free grammar cannot express it, so semantic analysis checks it with a symbol table.

A3 (b). Context-free grammars, recognised by pushdown automata: one stack, enough for nesting.

A4 (c). It is the Halting Problem.

A5 (b). At -00 every variable lives in a stack slot and is reloaded before every use; at -02 they stay in registers.

A6 (b). Soundness is “never wrong”; it says nothing about finding every possible improvement.

Solution: Lecture 2

A7 (b). Whitespace separates tokens (it is why `a b` is not `ab`) and is then dropped; no later phase needs it.

A8 (a). At the `-`, longest match compares `"-"` (1 character) with `"-"[0-9]+` matching `-1` (2 characters). Longest wins: `ID(x) INT(-1)`. The subtraction has vanished, which is exactly why number literals are unsigned in real lexers and negation belongs to the parser.

A9 (c). `<=` is 3 characters; longest match.

A10 (a). Star includes zero repetitions. A number rule must use `+`, or the lexer can return an empty token and loop forever.

A11 (b). Nesting to arbitrary depth requires counting, which a finite automaton cannot do. The other three are regular (a fixed pattern of characters).

A12 (b). Look before you consume: `peek()` decides, `read()` commits.

Solution: Lecture 3

A13 (d). Concatenation only adds one ϵ -edge from R_1 's accept to R_2 's start.

A14 (c). Each DFA state is a subset of the NFA's states.

A15 (b). That string is the witness; ϵ witnesses accepting-versus-non-accepting.

A16 (b). Clearing on leaving an accept keeps only the states since the most recent accept, so backtracking lands on the longest match.

A17 (c). Length is compared first; priority only breaks exact ties.

A18 (c). It records a dead end so that no later token re-explores it; that is what makes maximal munch linear.

Part B: understanding questions

Solution: B1

A parser reasons about structure in terms of *words*: “an *if*, a parenthesis, an expression.” If it worked on characters, every grammar rule would have to spell out letters, allow whitespace and comments between every pair of symbols, and re-describe how numbers and strings are written; the grammar would be huge and every rule would repeat the same clutter. Two concrete benefits of a separate lexer: every character-level nuisance (escapes, number formats, comments, tabs) is handled once in one small component, and the parser’s input alphabet becomes a few dozen token classes instead of the whole character set, which makes the parser both simpler and faster.

Solution: B2

If the lexer could produce signed literals, then by longest match the input `x-3` would lex as `ID(x) INT(-3)`: the minus sign is swallowed into the number, the subtraction disappears, and the parser sees two adjacent operands, which is a syntax error or, worse, silently wrong. The lexer cannot know whether a `-` is a binary minus or a unary sign, because that depends on what came *before*, which is the parser’s business. So literals are unsigned, `-` is always the operator token, and the parser decides from context whether it is negation or subtraction.

Solution: B3

The single principle: rule order only matters when two rules match the same longest prefix. A keyword such as `while` is also a valid identifier, so on the input `while` both rules match five characters and the tie is broken by order; the keyword must come first or it is unreachable. Two *different* keywords can never match the same text (each is a distinct literal string), so no tie between them ever arises and their relative order is irrelevant. For the same reason keyword rules never conflict with number or operator rules either.

Solution: B4

Because the end of a token is defined by the first character that cannot extend it, and the lexer only learns that by seeing that character. On `a==b`, after reading the first `=` the lexer cannot return `ASSIGN` yet: if the next character is another `=`, the token is `EQ`. It must look at the second character to decide. Likewise on `count` it needs to see the space (or the next non-letter) before it knows the identifier has ended. A hand-written lexer therefore peeks; a generated one reads ahead and backtracks, which is lookahead of arbitrary length made safe.

Solution: B5

A lexer generator runs subset construction *once*, at compiler-build time, on a specification that is fixed and written by the compiler author. Even a slow or large construction is paid a single time, and the resulting table is then used on millions of characters of source, where a DFA's constant-time-per-character behaviour is what matters. A regex library faces a new, user-supplied pattern at every call and cannot afford a potentially exponential preprocessing step before matching a single string; it therefore either simulates the NFA directly (building DFA states lazily) or backtracks over the NFA.

Solution: B6

The error state $\emptyset$ is a non-accepting state that every state moves to on any symbol for which the diagram has no arrow, and from which every symbol leads back to $\emptyset$. Adding it makes δ a total function, so the driver loop never has to ask “is there a transition?”; it just looks up the table. Since $\emptyset$ is absorbing and non-accepting, arriving there proves that no longer prefix can ever be accepted. That is precisely the signal Phase 1 of the scan-and-backtrack loop waits for: “stuck” means $\delta(q, c) = \emptyset$, and it is safe to stop reading ahead and backtrack to the last accept.

Solution: B7

Spec: $R_1 = \text{abc}$, $R_2 = (\text{abc})^*\text{d}$. Input: `abcbx`. Read-until-stuck consumes `abcb` and gets stuck on `x`; the state after `abcb` is not accepting, so “return what you have read” returns something that is not a token at all. The correct algorithm remembers the last accepting position (after `abc`) and backtracks to it, giving the characters `ab` back to the input, then returns `abc`. In general the last accepting state may be many characters before the point where the DFA gets stuck, which is why the stack and the unread operation exist.

Solution: B8

A finite automaton has a fixed number of states, hence a bounded amount of memory, but checking balance requires remembering how many parentheses are currently open, which is unbounded: with k states, the strings $(^{k+1}$ must send two different open-counts to the same state, and appending the matching number of `)` to one of them shows the automaton accepts an unbalanced string. Balanced structure is handled by the parser, whose machine is a pushdown automaton: the stack can hold an unbounded number of pending open brackets and pops one per close bracket.

Solution: B9

The text is the exact characters consumed; the value is what they *mean*. The string literal `"hi"` has text `"hi"` (four characters including the quotes) and value `hi`; the escape `"a\nb"` has a two-character escape in the text

and a real newline in the value. The integer 007 has text 007 and value 7; 0x1F has value 31. The parser and later phases compute with values (constant folding needs the number, string handling needs the decoded characters); the text matters only for error messages, so the lexer decodes once and passes the value.

Solution: B10

Whether a word is reserved is a property of the word's spelling, which is exactly the lexer's level: a literal string rule and priority over the identifier rule settle it in one place. If the lexer returned `ID("if")`, every grammar rule that mentions `if` would have to say “an identifier whose text is `if`,” and the parser would need to inspect token texts, which grammars cannot do; the parser would also have to reject `if` as a variable name in every other position. Keywords are lexical because deciding them needs no context, and the parser then works purely with token classes.

Solution: B11

Sound: whenever the analysis asserts a property (“this value is always 0,” “this code is never reached”), the property truly holds on every execution, so a transformation based on it never changes behaviour. *Conservative*: whenever the analysis cannot prove the property, it assumes the property may fail and leaves the code alone. The optimiser would like to know, say, whether a statement is ever executed; deciding that exactly is reachability, hence the Halting Problem, so it instead computes an over-approximation of the reachable set (following every branch as if it could be taken) and deletes only code that is unreachable even in that over-approximation.

Solution: B12

Inside a block comment the ordinary rules must be switched off: the characters `if (x) "` inside `/* ...*/` must not produce `IF`, `LP`, `ID` or start a string. A state (start condition) is a mode: entering `/*` moves the lexer into `<COMMENT>`, where only the comment-body rules and the `*/` rule are active; matching `*/` returns to `<YYINITIAL>`. A state changes *which rules are eligible* at all; longest match and priority then operate within the active set. The same mechanism handles string bodies, here-documents, and languages whose tokenisation depends on context.

Part C: problems

Solution: C1

1. Columns of `if (n>=10) x = x-1; i=1, f=2, sp=3, (=4, n=5, >=6, ==7, 1=8, 0=9, )=10, sp=11, x=12, sp=13, ==14, sp=15, x=16, -=17, 1=18, ;=19.`

class	value	position	class & value & position (cont.)
IF	–	(1,1)	ASSIGN, –, (1,14)
LP	–	(1,4)	ID, x, (1,16)
ID	n	(1,5)	MINUS, –, (1,17)
GE	–	(1,6)	INT, 1, (1,18)
INT	10	(1,8)	SEMI, –, (1,19)
RP	–	(1,10)	
ID	x	(1,12)	

Twelve tokens; whitespace produces none.

- After **if** the lexer must see the space at column 3 to know the identifier/keyword has ended (otherwise **iffy** would be an identifier). After the first = at column 7... no: at column 6 after >, it must see the = at column 7 to choose GE over a lone > (which is not even a token in this spec). Also at column 14 the = must be followed by a look at the space to rule out == if such a rule existed; with this spec the decisive lookaheads are the ones after **if**, after **n**, after >, after 10, and after each **x**: every variable-length or prefix-sharing token needs one character beyond its end.
- Only the first: **if** would become ID("if"), because both rules match two characters and the identifier rule would now win the tie. Nothing else changes: **n** and **x** are identifiers either way, and no other token is affected by rule order.
- x = - 1;** lexes as ID(x) ASSIGN MINUS INT(1) SEMI: at = the only match is ASSIGN (there is no == rule), at - only MINUS, then the space is skipped, then 1. The parser sees **x = - 1**, which is a legal assignment of a unary-minus expression; it accepts it and evaluates -1. The spacing is irrelevant to the lexer because whitespace merely terminates tokens.

Solution: C2

- With **exp = [eE][+-]?{digit}+**:

```
float = {digit}+ "." {digit}* {exp}?    // 3.14 3. 6.02e+23
      | "." {digit}+ {exp}?             // .5 .5e3
      | {digit}+ {exp}                  // 2e10 1E-3
```

Checks: 3 matches none of the three alternatives (no dot, no exponent) and is left to the integer rule; **e5** starts with a letter, so it is an identifier; . alone needs at least one digit on one side; **1.e** matches 1. by the first alternative and then **e** is a separate identifier.

- string = \" ([^\"\\n] | \\[ntr\"\\])* \":** an opening quote, then any number of either an ordinary character (anything except quote, backslash, newline) or a backslash followed by one of the allowed escape letters, then a closing quote. Excluding **\n** from the ordinary characters is what forbids spanning lines.
- linecomment = "//" [^\n]*:** the slashes, then anything up to (not including) the newline, which the whitespace rule then eats.

4. `ident = ( [a-zA-Z] | "_" + [a-zA-Z0-9] ) [a-zA-Z0-9]*`: either it starts with a letter, or it starts with one or more underscores that are followed by a letter or digit. The second branch guarantees that a run of underscores alone (`_`, `__`) is not an identifier, while `_x` and `__init1` are.
5. (a) `3.e`: lexes as `FLOAT(3.)` then `ID(e)`, not an error, because the lexer returns the longest prefix that is a token and moves on. (b) `"abc↓"` with a newline inside: the string rule cannot match across the newline, so the lexer returns whatever else matches at the opening quote; if no rule matches a lone quote, it is a lexical error (“unterminated string”), and if a catch-all error rule exists it becomes an `ERROR` token followed by `ID(abc)`. Either way it is not one string.

Solution: C3

1. Trace on `12+3`: `c = '1'` (digit, append), `c = '2'` (digit, append), `c = '+'`: not a digit, loop ends, but the `+` has already been *consumed* by `read()`. Return `INT(12)`. The next lexer call starts at `3` and returns `INT(3)`. Token stream: `INT(12) INT(3)`; the `PLUS` is lost.
2. “Consuming before checking” (the lookahead character is eaten). Fix:

```
StringBuilder sb = new StringBuilder();
while (isDigit(input.peek())) {      // look, do not consume
    sb.append((char) input.read());  // consume only what
    belongs
}
return new Token(INT, Integer.parseInt(sb.toString()));
```

3. `a==b` becomes `ID(a) ASSIGN ASSIGN ID(b)`: the lexer commits to `ASSIGN` on the first `=` without looking at the second. Fix: after reading `=`, `peek()`; if the next character is `=`, read it and return `EQ`, otherwise return `ASSIGN`. Principle: never return a token while a longer token might still be possible (longest match), which requires looking at least one character past the shorter candidate.
4. The string loop reads characters until it sees the closing quote. At end of input `read()` returns `-1` forever, which is never a quote, so the loop never ends (appending `(char)-1` each time). The loop must test for end of input *before* testing for the delimiter, and raise “unterminated string literal.”

Solution: C4

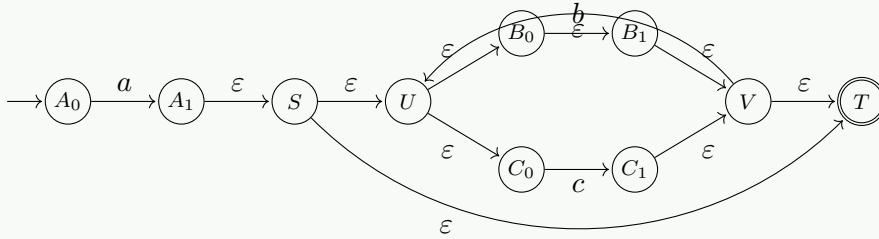
1. Because `{ident}` precedes `"if"`, the two-character tie on `if` goes to the identifier. `007` matches `{intlitt}` entirely (leading zeros are digits), value `7`. `3.5` has no float rule: `3` is `INT`, `.` is `DOT`, `5` is `INT`. Stream: `ID("if") ID("iffy") INT(7) INT(3) DOT INT(5)`.
2. Wrong or surprising: (i) `if` never lexes as a keyword; move `"if"` above `{ident}`. (ii) `3.5` is three tokens; add a float rule (Problem C2) above `{intlitt}` so that longest match picks it (order does not actually matter

here, since 3.5 is longer than 3, but placing it first documents the intent). (iii) 007 is silently accepted with value 7; if the language forbids leading zeros (or means octal by them), the rule should be "0" | [1-9][0-9]*, under which 007 lexes as three INT(0), INT(0), INT(7) tokens and the parser rejects adjacent integers. A fourth, minor point: `iffy` is correctly an identifier by longest match, which is worth stating to show the fix does not break it.

3. With the fixes (and `intlit = "0" | [1-9][0-9]*`): `IF ID("iffy") INT(0) INT(0) INT(7) FLOAT(3.5)`. With the original `[0-9]+` kept: `IF ID("iffy") INT(7) FLOAT(3.5)`.
4. `"else" { return ELSE; }` must be placed above `{ident}`; relative to `"if"` its position is irrelevant, since two distinct literals never tie.

Solution: C5

1. Parse: $R = E_1 \cdot E_2$, $E_1 = a$, $E_2 = E_0^*$, $E_0 = b \mid c$. Rules inside-out: symbol a (+2: A_0, A_1), symbol b (+2: B_0, B_1), symbol c (+2: C_0, C_1), union $b \mid c$ (+2: U, V), star (+2: S, T), concatenation (+0). Total 10.



2. $Q_0 = \{A_0\}$. On a : A_1 , closure $\{A_1, S, U, B_0, C_0, T\} = Q_1$ (accepting: contains T). Q_1 on b : $B_0 \rightarrow B_1$, closure $\{B_1, V, U, B_0, C_0, T\} = Q_2$ (accepting). Q_1 on c : $\{C_1, V, U, B_0, C_0, T\} = Q_3$ (accepting). Q_2 and Q_3 on b give Q_2 , on c give Q_3 (the only b/c edges leave B_0/C_0 , which both contain). Nothing has an a -edge except A_0 .

	a	b	c
Q_0 (start)	Q_1	$\emptyset$	$\emptyset$
Q_1 (acc)	$\emptyset$	Q_2	Q_3
Q_2 (acc)	$\emptyset$	Q_2	Q_3
Q_3 (acc)	$\emptyset$	Q_2	Q_3
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

3. Rule 1 marks every pair of an accepting state with Q_0 or $\emptyset$; $(Q_0, \emptyset)$: on a they go to $(Q_1, \emptyset)$, marked, so mark with witness a . Among $\{Q_1, Q_2, Q_3\}$: on every symbol all three go to the same state ($\emptyset$, Q_2 or Q_3 respectively), so Rule 2 never fires and all three are equivalent. Minimal DFA: $M_0 \xrightarrow{a} M_1$, $M_1 \xrightarrow{b,c} M_1$, accept M_1 , plus the error state. Two states (three with $\emptyset$): "an a , then any number of b 's and c 's."

4. NFA: $p_0 \xrightarrow{a,b} p_0$, $p_0 \xrightarrow{a} p_1 \xrightarrow{a,b} p_2$, accept p_2 (guess which a is the second-to-last). Subset construction: $\{p_0\} \xrightarrow{a} \{p_0, p_1\}$, $\{p_0\} \xrightarrow{b} \{p_0\}$; $\{p_0, p_1\} \xrightarrow{a} \{p_0, p_1, p_2\}$, $\xrightarrow{b} \{p_0, p_2\}$; $\{p_0, p_1, p_2\} \xrightarrow{a} \{p_0, p_1, p_2\}$, $\xrightarrow{b} \{p_0, p_2\}$; $\{p_0, p_2\} \xrightarrow{a} \{p_0, p_1\}$, $\xrightarrow{b} \{p_0\}$. Four states, accepting the two containing p_2 . *Lower bound*: the four strings aa, ab, ba, bb are pairwise distinguishable. For $u = xy$, $u \in L_2$ iff $x = a$, and $ub \in L_2$ iff $y = a$. So aa vs ba , aa vs bb , ab vs ba , ab vs bb are separated by the suffix ε (they differ in x), and aa vs ab , ba vs bb by the suffix b (they differ in y). Four pairwise distinguishable strings must reach four distinct states in any DFA for L_2 (if two reached the same state, every suffix would be accepted from both or neither). Hence every DFA has at least 4 states, and subset construction gave exactly that.

Solution: C6

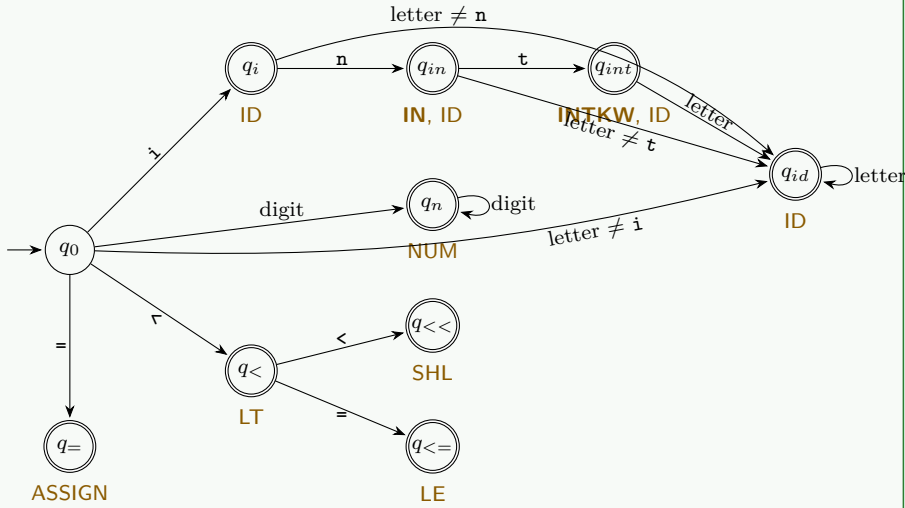
- From A : $A \rightarrow B, A'$; $A' \rightarrow B, A$; $B \rightarrow B, C$; $C \rightarrow B, A'$. Reachable: $\{A, A', B, C\}$. U is reachable from nothing, and C' only from U , so U and C' are unreachable. Remove both; the accepting set becomes $\{C\}$.
- Pairs among $\{A, A', B, C\}$, six of them. **Rule 1**: (A, C) , (A', C) , (B, C) marked, witness ε . **Rule 2, sweep 1**: (A, A') : on 0 both go to B ; on 1 they go to (A', A) , the same pair, unmarked; nothing. (A, B) : on 1 they go to (A', C) , marked; mark (A, B) with witness 1 (from A , 1 leads to A' , non-accepting; from B , to C , accepting). (A', B) : on 1 they go to (A, C) , marked; mark with witness 1. **Sweep 2**: (A, A') still leads only to itself and to (B, B) : nothing new. Stop.

	A	A'	B
A'	unmarked		
B	1	1	
C	ε	ε	ε

- Classes $\{A, A'\}$, $\{B\}$, $\{C\}$. Minimal DFA: $M_A \xrightarrow{0} M_B$, $M_A \xrightarrow{1} M_A$; $M_B \xrightarrow{0} M_B$, $M_B \xrightarrow{1} M_C$; $M_C \xrightarrow{0} M_B$, $M_C \xrightarrow{1} M_A$; start M_A , accept M_C . Language: binary strings that **end in 01**. (M_B = “the last symbol was 0,” M_C = “the last two were 01,” M_A = neither.)

Solution: C7

1.



2. Longest match first, then priority:

input	DFA path and stopping point	tokens
in	$q_0 \rightarrow q_i \rightarrow q_{in}$, EOF; tags IN, ID; IN listed first	IN
int	$\rightarrow q_{int}$, EOF; INTKW wins the tie	INTKW
integer	$\rightarrow q_{int} \xrightarrow{e} q_{id} \rightarrow \dots$, EOF; only ID	ID("integer")
i	q_i , EOF	ID("i")
in7	q_{in} , then 7: no letter edge, stuck; return IN; restart at 7	IN NUM(7)
«=	$q_{<} \rightarrow q_{<<}$, then =: stuck (no «= rule); return SHL; restart	SHL ASSIGN
<=<	$q_{<} \rightarrow q_{<=}$, then <: stuck; return LE; restart	LE LT
<<	$q_{<<}$ then <: stuck; SHL; restart at third <: $q_{<}$, EOF	SHL LT

3. IN and INTKW are different literal strings: no input can be the longest match of both, so they never tie and their order is irrelevant. SHL, LE and LT are also distinct literals of *different lengths*; when « is present, longest match already prefers it to <, and priority is never consulted. A pair whose order matters: "in" and [a-z]+: on the input in both match two characters, and whichever is listed first wins. In general, order matters only between a literal and a pattern that can match the same string, or between two overlapping patterns (e.g. [a-z]+ and [a-z0-9]+ on abc).

Solution: C8

Write S_w for the DFA state after reading w from q_0 . The relevant states: S_a (non-acc), S_{ab} (acc: R_1 complete, and R_2 has one **ab** repetition), S_{aba} (non-acc), S_{abab} (non-acc: R_1 is dead, R_2 still wants c); thereafter $S_{ababa} = S_{aba}$ and $S_{ababab} = S_{abab}$, since R_2 's loop has period two.

1. Positions 1..6 hold **a b a b a b**.

read (pos)	state after	acc.?	stack after the step
a (1)	S_a	no	$[q_0]$
b (2)	S_{ab}	yes	$[q_0, S_a]$
a (3)	S_{aba}	no	$[S_{ab}]$ (cleared on leaving an accept, then pushed)
b (4)	S_{abab}	no	$[S_{ab}, S_{aba}]$
a (5)	S_{abab}	no	$[S_{ab}, S_{aba}, S_{abab}]$
b (6)	S_{abab}	no	$[S_{ab}, S_{aba}, S_{abab}, S_{aba}]$
EOF			break

Phase 2: S_{abab} not accepting; pop S_{aba} (unread pos 6), pop S_{abab} (unread 5), pop S_{aba} (unread 4), pop S_{ab} (unread 3): accepting. Return $R1("ab")$, next token starts at position 3.

2. First token: 6 reads (all of the input) and 4 unreads. Second token (from position 3): reads $abab = 4$, backs up 2, returns ab . Third (from position 5): reads $ab = 2$, EOF, returns ab . Total reads $6 + 4 + 2 = 12$ for a 6-character file.
3. Token j starts at position $2(j-1)+1$ and reads to the end: $n - 2(j-1) = 2(k-j+1)$ characters. Summing over $j = 1..k$: $2(k + (k-1) + \dots + 1) = k(k+1) = \frac{n}{2}(\frac{n}{2} + 1) = \Theta(n^2)$.
4. Recall that the algorithm marks `hopeless[q][i]` in Phase 2 for every non-accepting configuration it pops through, where i is the position of the *next unread* character. For $n = 6$, Phase 1 of the first token ends with $q = S_{abab}$ and $i = 7$; Phase 2 marks $(S_{abab}, 7)$, pops to S_{aba} at $i = 6$ and marks it, pops to S_{abab} at $i = 5$ and marks it, pops to S_{aba} at $i = 4$ and marks it, then pops to S_{ab} at $i = 3$, which is accepting: stop. Marked: $(S_{abab}, 7)$, $(S_{aba}, 6)$, $(S_{abab}, 5)$, $(S_{aba}, 4)$.

Second token, from $(q_0, 3)$: not hopeless, read $a \rightarrow (S_a, 4)$; not hopeless, read $b \rightarrow (S_{ab}, 5)$; not hopeless, and S_{ab} is accepting so the stack is cleared and S_{ab} pushed; read $a \rightarrow (S_{aba}, 6)$: **hopeless**, break. S_{aba} is not accepting: pop S_{ab} , unread, $i = 5$, accepting: return ab . Three reads instead of four, and the lexer never re-explored the dead end. *Third token*, from $(q_0, 5)$: read a , read $b \rightarrow (S_{ab}, 7)$, EOF, return: two reads. Total $6 + 3 + 2 = 11$.

For $(ab)^k$ in general: the first token reads all n characters and marks every $(S_{aba}, \text{even } i)$ and $(S_{abab}, \text{odd } i)$ configuration beyond position 3. Every later token reads its own ab , then one more a , and finds (S_{aba}, i) already marked; the last token hits EOF after two reads. Total reads $n + 3(k-2) + 2 = 5k - 4 = O(n)$: linear, exactly as the lecture's bound $|Q| \cdot n$ promises, because each configuration (q, i) is pushed at most once over the whole run.

Solution: C9

1. Sound: adding zero changes no integer.
2. Unsound as stated: if x is $f()$ with a side effect (prints, modifies a global), dropping the call changes behaviour. Sound when x is a side-effect-free expression, e.g. a variable.
3. Unsound for negative x : $-3 / 2$ is -1 in C (truncation toward zero)

but $-3 \gg 1$ is -2 (arithmetic shift rounds toward $-\infty$). Sound if $x \geq 0$ is known.

4. Unsound if f has side effects or is non-deterministic (`rand()`, reading input): the two calls are observable. Sound if f is pure.
5. Sound: the condition is a compile-time constant false, so the body can never execute; no program can observe its absence.
6. Unsound if p and q may alias (point to the same location): the final stored value changes from 2 to 1. Sound if the compiler can prove $p \neq q$.
7. Sound under the stated precondition $n \geq 0$ and absence of overflow: both versions leave s increased by $1+2+\dots+n$ and n equal to 0; for $n = 0$ both do nothing. (If n could be negative the loop does nothing while the formula adds a positive amount: unsound without the precondition; and $n*(n+1)$ may overflow where the loop's running sum would not, so a careful compiler also needs a range bound.)

The unsound rewrites become sound given one extra fact each: (b) and (d) need “the expression/call has no side effects and is deterministic”; (c) needs “ x is non-negative”; (f) needs “ p and q do not alias.” All four facts are the kind an optimiser must *prove* conservatively (purity analysis, range analysis, alias analysis) and, failing proof, must assume false; that is Problem C10's theme.

Solution: C10

1. Suppose a decider D for RETURNS-ZERO exists. Given $\langle M, w \rangle$, construct

```
def f_Mw(x):
    simulate M on w      # ignore x
    return 0
```

- (computable from $\langle M, w \rangle$ by pasting). If M halts on w , then $f_{M,w}$ returns 0 on every input, in particular on at least one: $\langle f_{M,w} \rangle \in \text{RETURNS-ZERO}$. If M does not halt on w , $f_{M,w}$ never returns on any input, so it returns 0 on no input: $\langle f_{M,w} \rangle \notin \text{RETURNS-ZERO}$. Hence $\langle f_{M,w} \rangle \in \text{RETURNS-ZERO}$ iff $\langle M, w \rangle \in \text{HALT}_{TM}$, and “build $f_{M,w}$, run D ” would decide HALT_{TM} . Contradiction; RETURNS-ZERO is undecidable.
2. Also undecidable, by the same construction: $f_{M,w}$ returns 0 on *every* input iff M halts on w (if M never halts, $f_{M,w}$ returns 0 on no input, so certainly not on every input). Both the existential and the universal versions reduce from Halting.
 3. Constant propagation tracks, for each expression, “definitely this constant” or “unknown.” On the `if` branch it sees the literal 0. On the other branch it sees $g(x) - g(x)$: to fold this to 0 it must know that both calls return the same value, which requires knowing that g is pure (no side effects, deterministic) and that it terminates; a conservative analysis that cannot prove purity treats each call as an unknown value, so the difference is “unknown,” the return value of f is the join of 0

and “unknown,” i.e. unknown, and the call sites are not replaced. The optimiser is right to hesitate: if `g` reads a global that changes between the calls, the difference is not 0. Only with a purity proof for `g` would the fold be sound.

4. **Scanning identifiers:** Type 3; a fixed pattern of characters, finite automaton. **Matching brackets:** Type 2; needs an unbounded counter, one stack suffices. **Right number of arguments:** Type 1 in the hierarchy (it relates a call to a declaration elsewhere, which a CFG cannot express) and implemented as a symbol-table check in semantic analysis. **Variable ever read after its last write:** Type 0; exact liveness on real control flow is reachability, hence undecidable; compilers compute a conservative over-approximation. **String literals with escapes:** Type 3; escapes are a fixed two-character pattern, no counting. **Rejecting nested comments deeper than 3:** Type 3, surprisingly: the depth is bounded by a constant, so a finite automaton with a state per depth (0, 1, 2, 3, and “too deep”) suffices; it is unbounded nesting that is not regular.